

南京信息工程大学 实验（实习）报告

实验（实习）名称 线性分组码编译码 日期 2026.5.19 得分

指导教师 崔琦 班级 24 奇安信 1 班 姓名 孙涛 学号 202483950014

一、实验目的

本次实验以 $(5, 2)$ 线性分组码为具体例子，要求实现根据生成矩阵 G 进行编码，使用校验矩阵 H 进行检查，以及计算得到标准阵列并使用标准阵列译码（实现对于接受序列的检错纠错）。

二、实验原理

对于 (n, k) 线性分组码，即把 k 位信息位通过添加 $r=n-k$ 位冗余位形成 n 位序列后再传输，遵循编码规则 $c=mG$ ，在本例中因为信息位只有两位，所以 m 就是 4 种情况，00, 01, 10, 11；然后再矩阵乘以 G 就得到理论上正确传输应该得到的 4 种长为 5 的编码。如何选择 G 是我在实验中遇到的问题，在经过查询与学习后，一个好的生成矩阵 G 应该能够唯一纠正一位错误，即在一致校验矩阵 H 中不会出现重复列，也就使得每一个单错误的 syndrome 唯一，后文再详细解释，这里我选择的生成矩阵是：

$$G = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

有了生成矩阵 G 矩阵乘就能算出 0 出错的情况下的 4 种编码（我在 c 语言代码中直接使用 & 运算和 ^ 运算来实现矩阵乘），即 {00000, 01011, 10101, 11110}，这同时也是标准阵列译码的第一行。这里的 4 个编码是满足互相相加（模 2）后任在子集里的，这说明是线性的。

对于一致校验矩阵 H ，是 $r \times n$ 的一个矩阵，通过把 G 中的 P 转置后在补上单位阵得到，在本例中的一致校验矩阵是：

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

一致校验矩阵的作用是检查码字是否合法，合法码字 c 应该满足 $H \cdot c^T = 0$ 。不难检验上文给出的 4 个 c 都是合法的，而如果传输中出现错误，那么显然 $H \cdot c^T \neq 0$ ，这个计算的结果就是 syndrome（伴随式），有了伴随式是如何检验错误出在 5 位中的哪一位或者哪几位，这里要提到错位向量 e ， e 是一个 1 行 5 列的矩阵，并且传输无错的情况下是全 0，传输错在第几位那么第几位就是 1，理由是对于合法码字 c ，出错码字 $c_1 = c + e$ （模 2），其实就是逐位做异或，0 和任何数异或都是任何数，而 1 和 0 异或把 0 变成了 1，1 和 1 异或把 1 变成了 0，那么在此基础上我们就可以得到 $H \cdot c_1^T = H \cdot c^T + H \cdot e^T$ ，而 $H \cdot c^T = 0$ ，所以 S （伴随式） $= H \cdot e^T$ ，这说明 syndrome 只有错误向量有关，而与发送的原本码字是什么无关。那么我们得到 syndrome 就能得到 e ，得到 e 就能根据接受序列算出原序列（通过模 2 加）。

对于使用标准阵列译码表来译码，标准阵列译码表是一个 2^{**n} 行， 2^{**k+1} 列的一个矩阵，其中第一列是由错误向量 e 构成，也称为陪集首，即从 0 错误（00000）遍历到全错误（11111），而后面的 2^{**k} 列就是拿错误向量加上合法码字得到的结果，这样当接收方得到一个任意码字，接收方只需要对照这个表，找到它的位置，那么它所在列的第一行就是发送方想要传输的内容，因为第一行是 0 错误的。那么我在写实验的时候就想到这个问题，既然有标准阵列表为什么要引入 syndrome 的概念。经过查询和学习，原因在于本例中码字仅长 5，表一共只有 32 行，而如果信息位增长，校验位也增长，标准阵列表会很大。而使用 syndrome，在每次接收到一个码字 r 的时候，都用 $H \cdot r^T$ ，即 $H \cdot e^T$ 得到伴随式 S ，这里的 S 是 $(n-k) \times 1$ 的，本例中是一个 3 位的，这是比对 syndrome 表（下述），就能直接得到错误向量 e ，然后用 $e + r$ 就得到结果。

在写这一步的过程中我遇到的又一个问题是为什么最多 8 种的 syndrome 能对应上 32 中 e ，在查询和学习后，对于本例 5 位长，总共有 32 种可能，而合法码字只有 4 个，产生 8 个陪集，就是说 8 中 syndrome 区分 8 种陪集，也就是说不同的错误可能会对应同一个 syndrome。而我们默认出现越少错误的可能性越大，那么就能得到 syndrome 表，即 $S=000$ ，对应 $e=00000$ ，直到 $S=111$ ，对应 $e=01100$ 。

到此，本次实验所有的原理均已展现。

三、实验内容

1. 定义生成矩阵 G ，并得到一致校验矩阵 H 。
2. 把所有可能的信息位组合（本例中 4 种）并计算对应码字。
3. 检验 $H \cdot c^T$ 是否等于 0，即第二步中得到的码字是否合法（我在代码中省略了这步）。
4. 生成所有的伴随式及对应的陪集首（须关联起来）。
5. 译码全过程，即对得到的一个码字 r 先计算伴随式，然后找对应错误向量 e ，然后计算发送方想发送的码字，最后提取信息位。

四、实验代码和结果截图

实验代码中的 G 和 H 我是直接写入的， H 想通过计算实现也不难。

信息位与 G 计算码字代码如下： `void encode(int *message, int *codeword) {`

```
for (int i = 0; i < N; i++) {
```

```
    codeword[i] = 0; // 初始化码字为 0，因为 0 异或任何数都等于任何数
```

```
    for (int j = 0; j < K; j++) {
```

```
        codeword[i] ^= (message[j] & G[j][i]); // 信息位的第 j 位与矩阵 G
```

```
        的第 j 行第 i 列按位异或，得到码字的第 i 位
```

```
    }
```

```
}
```

```
} // 对于真正要串数的 2 位信息位，矩阵乘 G，得到 5 位码字
```

得到标准阵列译码表的代码如下： `void init_coset_table(CosetTable *table) {`

```
// 生成所有  $2^R=8$  种伴随式及其对应的陪集首
```

```
// 陪集首按重量从小到大排列（标准阵列第一列）
```

```
// 重量 0（全零错误）
```

```
int syndrome0[R] = {0, 0, 0};
```

```
int leader0[N] = {0, 0, 0, 0, 0};
```

```
for (int i = 0; i < R; i++) table[0].syndrome[i] = syndrome0[i];
```

```
for (int i = 0; i < N; i++) table[0].coset_leader[i] = leader0[i]; // 当只有 0 位错
```

```
误时，伴随式是 000，陪集首是 00000
```

```

// 重量 1 的错误
// 根据 H 计算:  $S = H \cdot e^T$  (3,5*5,1)
int syndromes1[5][R] = {
    {1, 0, 1}, // 错误在第 0 位  $e=[1,0,0,0,0]$ 
    {0, 1, 1}, // 错误在第 1 位  $e=[0,1,0,0,0]$ 
    {1, 0, 0}, // 错误在第 2 位  $e=[0,0,1,0,0]$ 
    {0, 1, 0}, // 错误在第 3 位  $e=[0,0,0,1,0]$ 
    {0, 0, 1} // 错误在第 4 位  $e=[0,0,0,0,1]$ 
};

int leaders1[5][N] = {
    {1, 0, 0, 0, 0},
    {0, 1, 0, 0, 0},
    {0, 0, 1, 0, 0},
    {0, 0, 0, 1, 0},
    {0, 0, 0, 0, 1}
};

for (int i = 0; i < 5; i++) {
    for (int j = 0; j < R; j++) table[i+1].syndrome[j] = syndromes1[i][j];
    for (int j = 0; j < N; j++) table[i+1].coset_leader[j] = leaders1[i][j];
} // 截至这里 8 种伴随式已经确定了 6 种，且每一个伴随式都找到了对应陪
集首

```

```

// 重量 2 的错误 (剩余 2 种伴随式)
//  $S=[1,1,0] \rightarrow$  错误  $e=[0,0,1,1,0]$  (第 3 和第 4 位)
//  $S=[1,1,1] \rightarrow$  错误  $e=[0,1,1,0,0]$  (第 1 和第 2 位)
// 这里的 e 是根据 s 倒推出来的

```

```

int syndromes2[2][R] = {
    {1, 1, 0}, //  $S=[1,0,1]$ 
    {1, 1, 1} //  $S=[1,1,1]$ 
};

```

```

int leaders2[2][N] = {
    {0, 0, 1, 1, 0},
    {0, 1, 1, 0, 0}
};
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < R; j++) table[i+6].syndrome[j] = syndromes2[i][j];
    for (int j = 0; j < N; j++) table[i+6].coset_leader[j] = leaders2[i][j];
} //截至这里 8 种伴随式已经确定了 8 种，且每一个伴随式都找到了对应陪
集首
}

计算伴随式的代码如下：void compute_syndrome(int *received, int
*syndrome) {
    for (int i = 0; i < R; i++) {
        syndrome[i] = 0; //初始化伴随式为 0，因为 0 异或任何数都等于任何数
        for (int j = 0; j < N; j++) {
            syndrome[i] ^= (received[j] & H[i][j]); //实质是拿 5 位码字矩阵乘
H 的转置，最终得到的伴随式
        }
    }
}

```

输入输出以及根据伴随式找错误向量以及提取信息位的代码不再赘述。

实验截图如下：

```

Active code page: 65001
PS C:\Users\23059\Desktop\信息论\新建文件夹> gcc linear_code.c -o linear_code.exe
PS C:\Users\23059\Desktop\信息论\新建文件夹> .\linear_code.exe
===== (5,2)线性分组码实验 =====
生成矩阵G:
10101
01011

监督矩阵H:
10100
01010
11001

所有信息码字映射:
信息 00 -> 码字 00000
信息 01 -> 码字 01011
信息 10 -> 码字 10101
信息 11 -> 码字 11110

===== 用户输入译码 =====

请输入5位接收码字（用空格分隔，如：1 0 1 1 0），输入-1退出：
0 1 0 1 1

接收码字：01011
译码结果：01011
提取信息：01

请输入5位接收码字（用空格分隔，如：1 0 1 1 0），输入-1退出：
0 1 0 1 0

接收码字：01010
译码结果：01011
提取信息：01

请输入5位接收码字（用空格分隔，如：1 0 1 1 0），输入-1退出：
1 1 1 1 1

接收码字：11111
译码结果：11110
提取信息：11

请输入5位接收码字（用空格分隔，如：1 0 1 1 0），输入-1退出：
1 0 0 0 0

```

五、实验心得

本次实验实现了线性分组码的编码以及使用标准阵列表译码的全过程，在本次实验及代码的编写过程中我加深了对应编码译码原理的理解，通过查询资料和学习解决了自己的不少疑问，在实验原理的部分中已详细书写。